Case Study: Search Engine Query Processing Using Binary Search

CSA0613-Design and Analysis of Algorithms

Course Outcome: CO3_AT2

Student Name: A. Bhavya(192424417)

Q3. Case Study: Search Engine Query Processing (Binary Search)

A search engine maintains a sorted index of keywords and must retrieve results quickly. Analyse how Binary Search improves search efficiency. Identify key requirements such as sorted data and fast retrieval. Apply divide and conquer principles to explain the process. Analyse time complexity and performance. Design a solution and justify why Binary Search is optimal in this context.

1. Problem Overview

Search engines maintain a large collection of keywords in a sorted order to retrieve information quickly. Binary Search is used to reduce the search time and improve efficiency. The algorithm divides the data into smaller parts until the required keyword is found.

2. Objectives of the Study

- Understand the role of Binary Search in search engines.
- Improve search efficiency.
- Retrieve results quickly from large datasets.
- Apply the divide-and-conquer technique.
- Analyze performance and complexity.

3. System Requirements

The following requirements are necessary for implementing Binary Search:

- Keywords must be stored in sorted order.
- Fast retrieval of data is required.
- Efficient memory utilization.
- Quick comparison of elements.

- Support for large datasets.

4. Working Principle of Binary Search

Binary Search follows the divide-and-conquer approach.

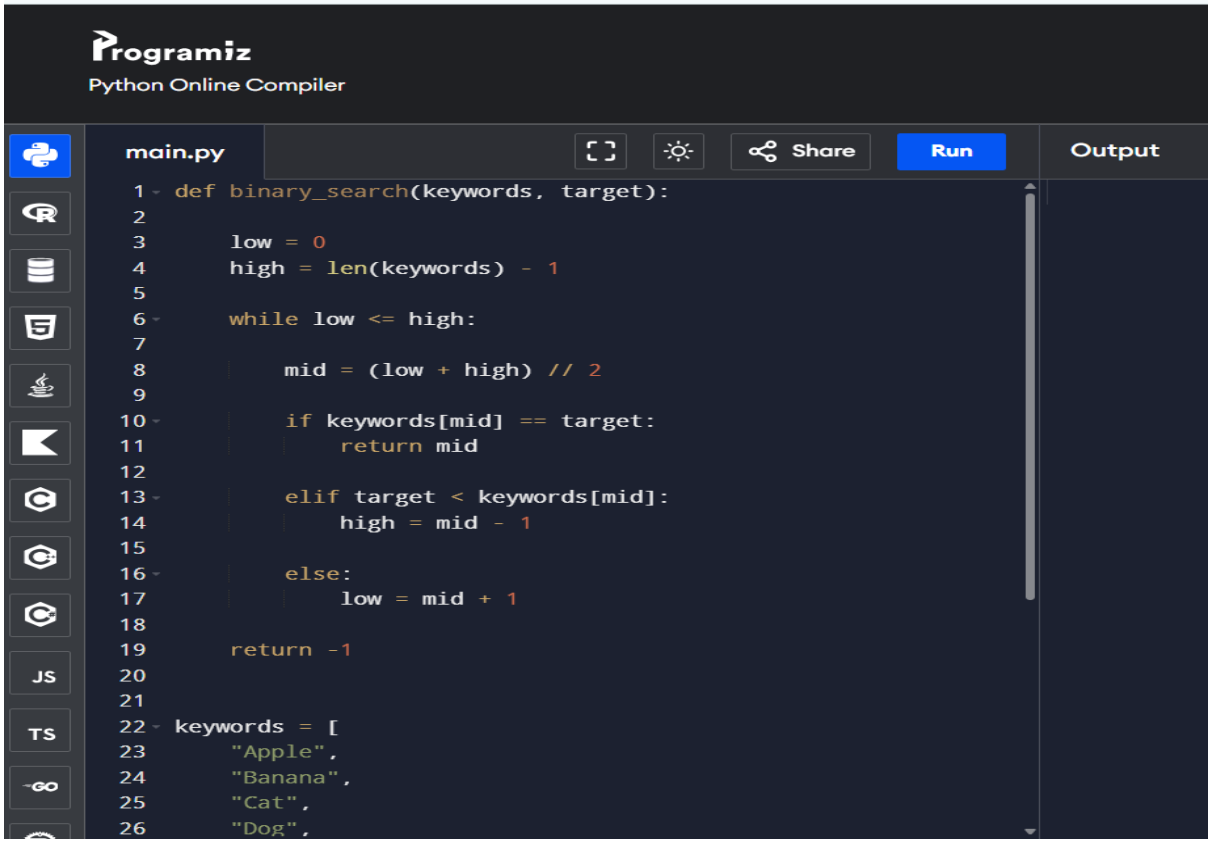
Steps:

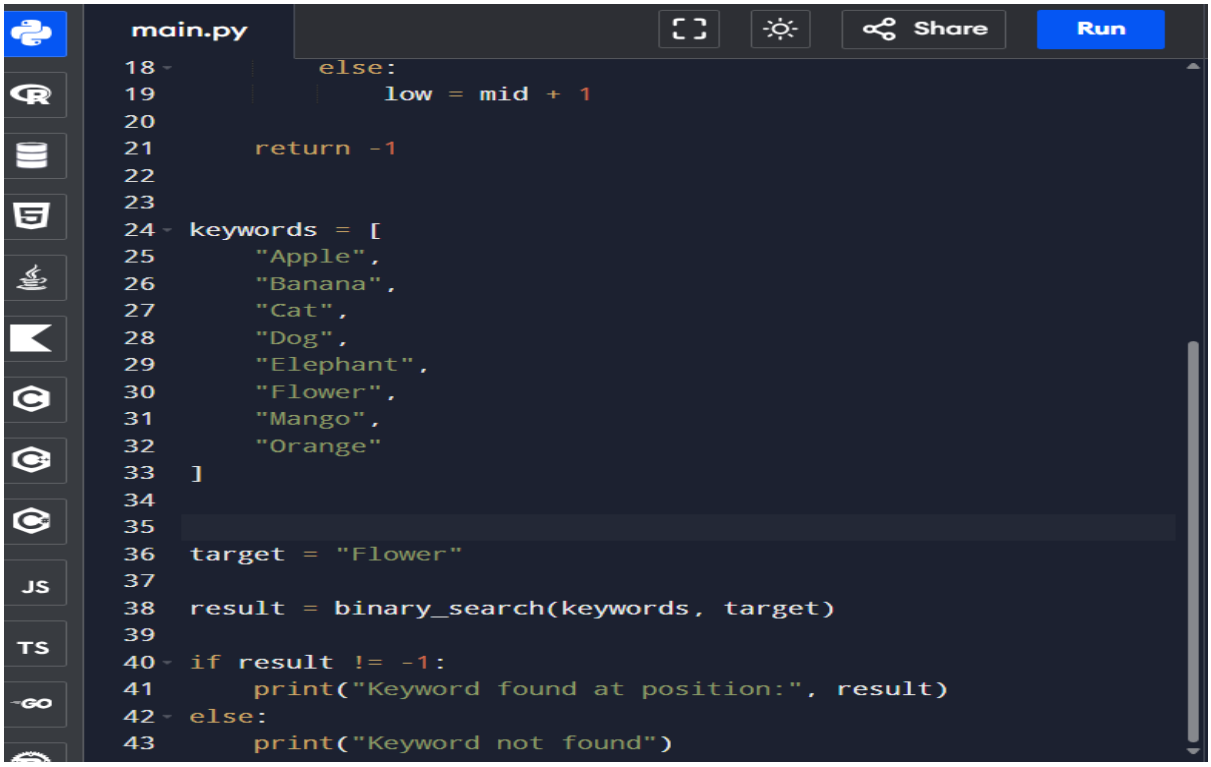
1. Find the middle element.
2. Compare it with the search keyword.
3. If both are equal, return the result.
4. If the keyword is smaller, search the left half.
5. If the keyword is larger, search the right half.
6. Repeat the process until the element is found.

5. Algorithm Design:

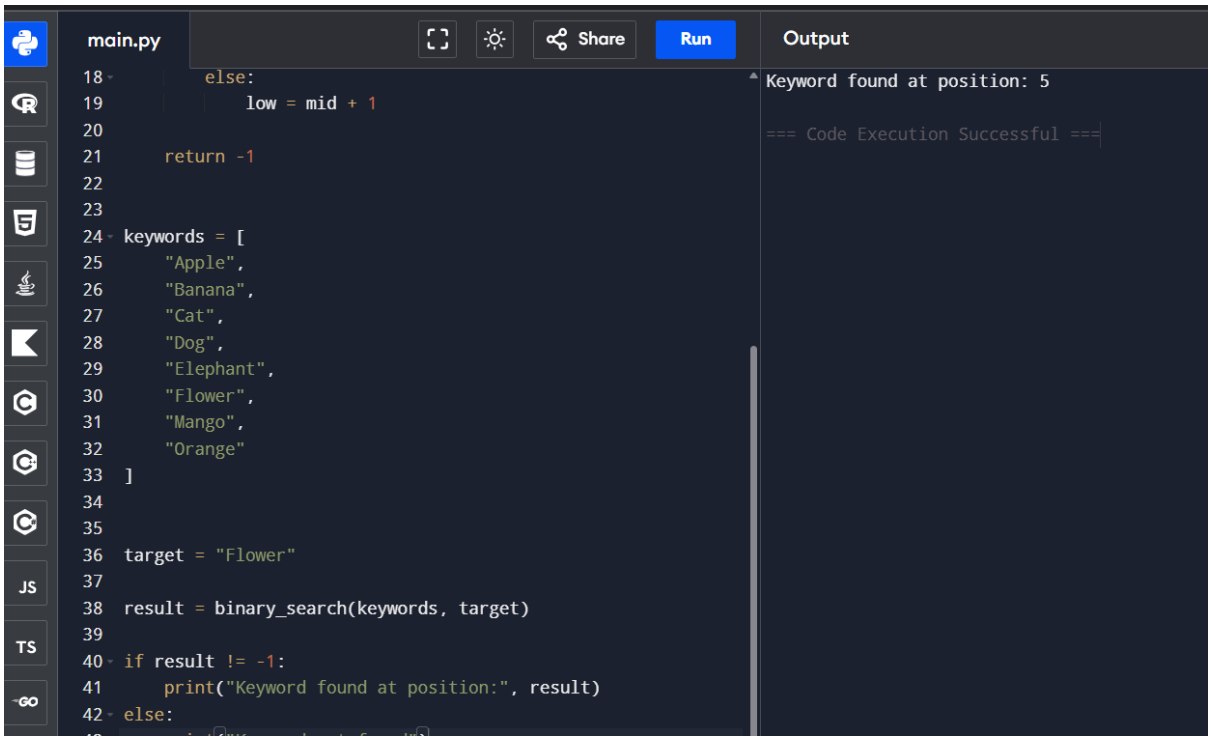
```
BinarySearch(arr, low, high, key)
while low <= high
   mid = (low + high) / 2
   if arr[mid] == key
      return mid
   else if key < arr[mid]
      high = mid - 1
   else
      low = mid + 1
return -1
```

SOURCE CODE:


Programiz
Python Online Compiler
main.py
Share
Run
Output
def binary_search(keywords, target):
low = 0
high = len(keywords) - 1
while low <= high:
mid = (low + high) // 2
if keywords[mid] == target:
return mid
elif target < keywords[mid]:
high = mid - 1
else:
low = mid + 1
return -1
keywords = [
"Apple",
"Banana",
"Cat",
"Dog",


main.py
Share
Run
else:
low = mid + 1
return -1
keywords = [
"Apple",
"Banana",
"Cat",
"Dog",
"Elephant",
"Flower",
"Mango",
"Orange"
]
target = "Flower"
result = binary_search(keywords, target)
if result != -1:
print("Keyword found at position:", result)
else:
print("Keyword not found")

OUTPUT:



Complexity Evaluation:

Case	Time Complexity
Best Case	O(1)
Average Case	O(log n)
Worst Case	O(log n)

Space Complexity :O(1)

Conclusion:

Binary Search is an efficient algorithm for search engine query processing. By applying the divide-and-conquer technique, it improves search efficiency and reduces execution time. Therefore, Binary Search is an optimal solution for search engines.